

**INSTITUTO FEDERAL DE EDUCAÇÃO, CIÊNCIA E TECNOLOGIA
CAMPUS CAMPINA GRANDE
ENGENHARIA DA COMPUTAÇÃO**

**FABRÍCIO MORENO
MARIA EDUARDA
LUCAS DARIS
YNNAYRON JUAN**

RELATÓRIO TÉCNICO: MESA LABIRINTO CONTROLADA POR ESP32

CAMPINA GRANDE - PB

2026

Este relatório descreve a arquitetura de software, o fluxo de execução das tarefas, a lógica de funcionamento do hardware e os componentes de software utilizados no projeto final da disciplina de Sistemas Embarcados.

1. Arquitetura de Software e Fluxos FreeRTOS

O sistema foi modelado utilizando uma arquitetura multitarefa executada sobre o sistema operacional em tempo real FreeRTOS. As tarefas foram projetadas com foco em modularidade, separando as responsabilidades de entrada de comandos, controle físico, sensoriamento e status do sistema.

Resumo das Tarefas (Tasks)

joystick_read_task

- Prioridade: 3
- Tamanho da Stack: 2048 bytes
- Período: 10 ms
- Responsabilidade: Lê os eixos analógicos do joystick (via conversor analógico-digital - ADC) e o estado do botão digital. Aplica algoritmos de média móvel e debounce.

servo_control_task

- Prioridade: 2
- Tamanho da Stack: 2048 bytes
- Período: 20 ms
- Responsabilidade: Lê as coordenadas suavizadas do joystick do estado global, aplica a zona morta (deadzone), calcula o sinal PWM correto (LEDC) e realiza uma interpolação exponencial para suavização do movimento.

mpu6050_task

- Prioridade: 2
- Tamanho da Stack: 4096 bytes
- Período: 10 ms
- Responsabilidade: Inicializa e configura o sensor inercial MPU6050, executa a calibração inicial dos offsets do giroscópio, calcula a orientação angular (pitch e roll) usando um algoritmo de Filtro Complementar e transmite periodicamente esses dados em formato JSON através da porta serial (UART).

status_task

- Prioridade: 1
- Tamanho da Stack: 2048 bytes
- Período: 500 ms
- Responsabilidade: Executa um padrão visual de piscar no LED de status após o boot do sistema, sinaliza que a mesa está pronta para uso e imprime logs informativos na porta serial.

Sincronização e Comunicação (Mutex)

Para garantir a consistência de dados em um microcontrolador de núcleo único e evitar condições de corrida (onde tarefas distintas tentam acessar ou modificar a mesma região da memória de forma concorrente), o sistema implementa:

1. Estrutura Global de Dados (system_data_t global_data): Concentra todas as variáveis de estado do sistema, como leituras brutas de joystick, ângulos filtrados de pitch/roll, limites de PWM dos servos, estados do botão.

2. Semáforo Mutex (data_mutex): Criado na inicialização através de xSemaphoreCreateMutex(). Qualquer tarefa que necessite escrever ou ler dados do estado global deve primeiramente tomar a chave de acesso (Mutex) utilizando xSemaphoreTake, copiar os dados de forma rápida e segura ou alterá-los, e devolver a chave imediatamente com xSemaphoreGive.

Fluxo de Dados

- As tarefas joystick_read_task (eixos e gatilho de calibração), mpu6050_task (ângulos e leitura bruta de sensores) atuam como produtoras de dados, gravando suas informações na estrutura global protegida.
- As tarefas servo_control_task (leitura do joystick para os motores), status_task (logs de depuração) atuam como consumidoras de dados, consultando o estado global após obterem o Mutex de sincronização.

2. Explicação do Funcionamento do Sistema

O funcionamento geral do sistema embarcado ocorre de forma cíclica e pode ser dividido nos seguintes subprocessos:

AQUISIÇÃO DE ENTRADA: JOYSTICK ANALÓGICO

- **Média Móvel (Filtro Passa-Baixas):** Os sensores analógicos do joystick apresentam flutuações e ruído elétrico natural. A tarefa joystick_read_task mantém buffers de 16 posições para os eixos X e Y. A cada ciclo de 10 ms, a leitura mais antiga é descartada e a nova é armazenada. O valor utilizado pelo sistema é a média aritmética desse buffer, o que elimina oscilações e tremores finos sem introduzir atraso perceptível no controle.

- **Debounce do Botão:** O botão do joystick (SEL) envia um sinal digital que passa por um debounce por software. Ele exige que o sinal de clique permaneça ativo por um tempo mínimo (entre 50ms e 500ms) para disparar a flag de recalibração rápida, evitando que múltiplos cliques falsos sejam interpretados por conta do ruído mecânico dos contatos elétricos.

CONTROLE DINÂMICO: SERVOMOTORES

- **Modulação por Largura de Pulso (PWM):** O ESP32-C6 controla a posição dos servomotores utilizando o periférico de hardware LEDC configurado em 50 Hz (período padrão de 20 ms para servomotores). A resolução do temporizador de PWM é de 13 bits (valores de controle variando de 0 a 8191).

- **Limites Físicos de Curso:** Para prevenir danos mecânicos e não forçar as engrenagens dos servomotores nos extremos do labirinto, a largura do pulso de PWM é restringida por software a valores equivalentes ao intervalo de trabalho físico do projeto (aproximadamente entre 1 ms e 2 ms de pulso). Esses valores correspondem ao intervalo digital de 410 (0 graus de inclinação) a 820 (180 graus de inclinação).

- **Zona Morta (Deadzone):** Adicionou-se uma tolerância de 50 pontos em torno do valor de repouso central do joystick (2048). Se o joystick estiver próximo ao centro físico, o sistema força a leitura para o centro nominal, garantindo que os motores fiquem completamente imóveis se o usuário soltar o comando.

- **Filtro Exponencial de Movimento:** Os movimentos do servomotor são suavizados usando uma interpolação matemática:

$$\text{Posição Atual} = \text{Posição Atual} + (\text{Posição Alvo} - \text{Posição Atual}) * 0.60$$

Esse cálculo faz com que a mesa se desloque de forma fluida e amortecida, evitando acelerações abruptas que fariam a esfera de metal saltar das canaletas do labirinto.

MEDIÇÃO DE ORIENTAÇÃO: SENSOR MPU6050

- **Calibração Inicial:** Antes de iniciar o monitoramento, a tarefa do MPU6050 realiza a leitura de 200 amostras consecutivas da velocidade angular com a mesa parada para extrair e armazenar o desvio padrão (offset) do giroscópio. Para evitar que o mecanismo de proteção contra travamento do processador (Task Watchdog) seja acionado, a tarefa cede temporariamente a CPU ao sistema operacional a cada 20 amostras coletadas.

- **Filtro Complementar:** Para estimar a inclinação da mesa nos eixos pitch (eixo X) e roll (eixo Y), os dados brutos são processados através de um filtro complementar:

$$\hat{\text{Ângulo}} = 0.96 * (\hat{\text{Ângulo Anterior}} + \text{Velocidade do Giroscópio} * \text{Intervalo de Tempo}) + 0.04 * \hat{\text{Ângulo do Acelerômetro}}$$

O giroscópio fornece velocidade angular de forma rápida e precisa (livre de ruídos de aceleração externa), mas acumula um erro no tempo (drift). O acelerômetro calcula a gravidade estática, não sofre desvio no tempo, mas oscila devido à movimentação física. A combinação ponderada de 96% giroscópio e 4% acelerômetro resolve ambos os problemas, entregando um sinal limpo e estável.

TELEMETRIA E VISUALIZAÇÃO EM TEMPO REAL (GÊMEO DIGITAL)

- A cada 100 ms (frequência de 10 Hz), a tarefa de monitoramento empacota os dados da mesa (ângulos da mesa, leituras analógicas brutas, aceleração tridimensional e estados lógicos) em formato JSON e os imprime no terminal serial (UART).
- Um script em linguagem Python é executado no computador, capturando as strings JSON provenientes da porta serial e inserindo-as em tempo real em um banco de dados de séries temporais local chamado InfluxDB.
- Um painel do Grafana integrado à biblioteca gráfica Web 3D (Three.js) puxa os dados do InfluxDB e move um modelo digital geométrico tridimensional (arquivo mesa.glb) que espelha perfeitamente cada alteração física de inclinação sofrida pela mesa física do labirinto.

3. Bibliotecas e Componentes Utilizados

O desenvolvimento do firmware no ESP32-C6 emprega as seguintes APIs, bibliotecas e componentes externos:

Drivers de Hardware do ESP-IDF (Componentes IDF)

- esp_adc/adc_oneshot.h: Camada de abstração de hardware que permite a configuração da resolução e a execução de leituras analógicas rápidas e controladas dos eixos do joystick.
- driver/gpio.h: Permite o controle elétrico digital dos pinos de entrada e saída, incluindo configuração de resistores de elevação (pull-up) e níveis lógicos.
- driver/ledc.h: Utilizado para programar o hardware dedicado a modulação por largura de pulso (PWM) do microcontrolador com alta estabilidade, comandando as posições angulares dos servomotores.
- driver/i2c.h: Gerencia a comunicação serial síncrona I2C em modo mestre (Master) a 400 kHz, necessária para a leitura e escrita nos registradores internos do sensor inercial MPU6050.

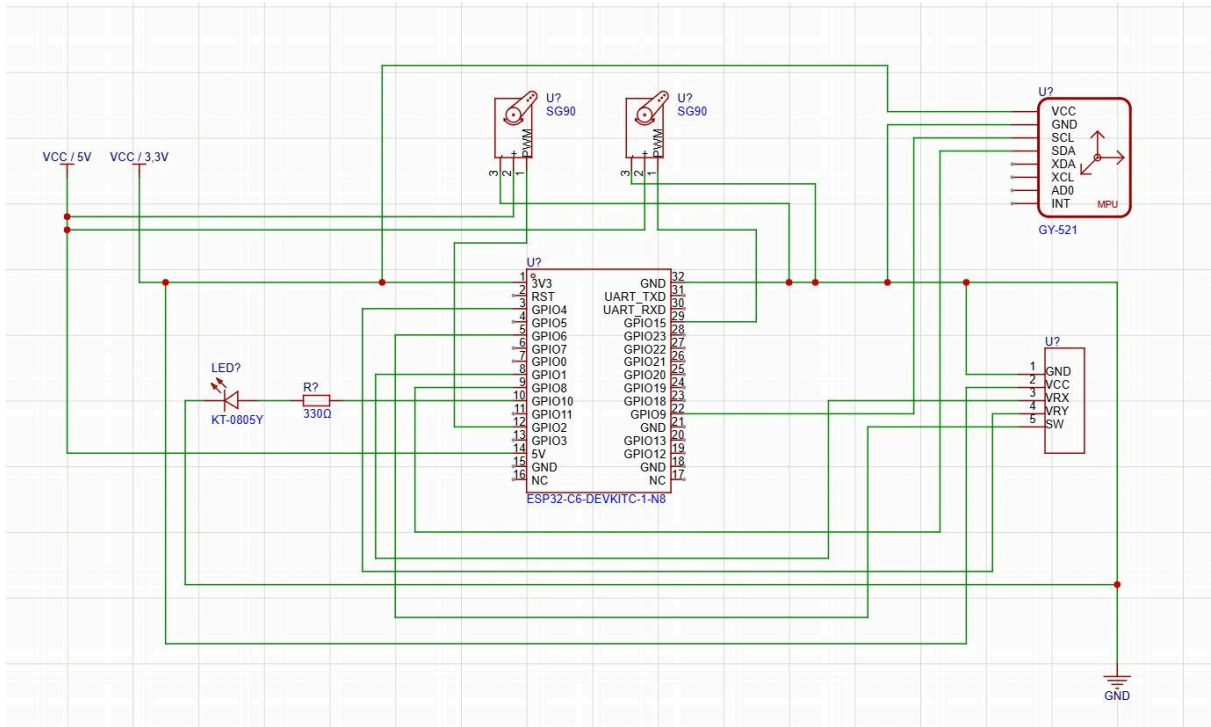
Bibliotecas de Sistema e Computação

- freertos/FreeRTOS.h e freertos/task.h: Núcleo do sistema operacional que possibilita a criação de threads de execução paralela (tarefas), gerencia a prioridade do agendador e fornece rotinas de atraso não bloqueante (vTaskDelay).
- freertos/semphr.h: Fornece as definições necessárias para a manipulação de semáforos e semáforos mutex, usados para blindar os recursos e variáveis globais contra acessos simultâneos corrompidos.
- math.h (Biblioteca Padrão C): Implementa funções matemáticas avançadas usadas no processamento vetorial do sensor de orientação (como raiz quadrada sqrtf e arco tangente atan2f).

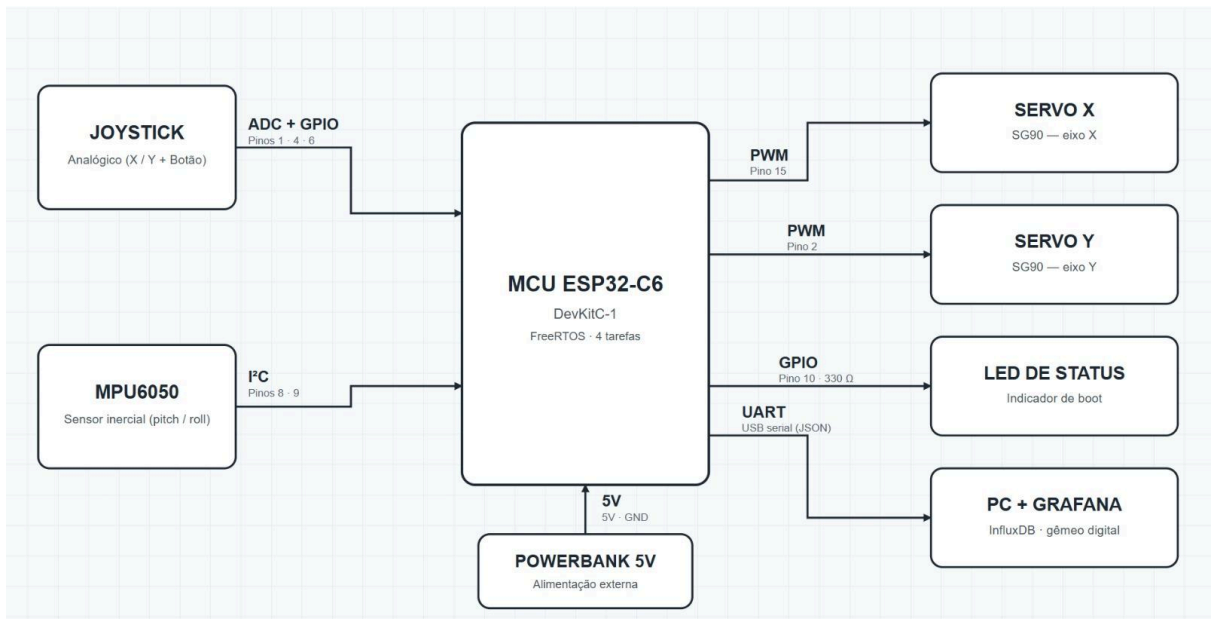
Tecnologias e Bibliotecas Externas (Apresentação e Banco de Dados)

- influxdb_client (Python Client): Biblioteca de integração usada no script Python para conexão segura e postagem síncrona via HTTP no banco de dados InfluxDB.
- Three.js (Via CDN esm.sh): Motor de renderização gráfica JavaScript leve para criação de cenas, iluminação, câmeras e controle de malhas 3D no navegador web.
- GLTFLoader.js e OrbitControls.js: Plugins do ecossistema Three.js utilizados para carregar arquivos 3D comprimidos em formato GLTF/GLB e para permitir a manipulação interativa da câmera orbital em torno da mesa 3D com o mouse.

4. DIAGRAMA ESQUEMÁTICO



5. DIAGRAMA EM BLOCOS DO SISTEMA



6. DIFICULDADES ENCONTRADAS

Durante o desenvolvimento do projeto, a equipe enfrentou alguns desafios técnicos complexos que exigiram pesquisa bibliográfica e testes experimentais detalhados:

Integração e Ciclo de Atualização do Grafana

Uma das maiores dificuldades foi entender como o Grafana renderiza e atualiza painéis personalizados com scripts de terceiros.

Compreensão e Modelagem do Sensor MPU6050

Outro obstáculo significativo foi o estudo e a calibração do sensor inercial MPU6050. Foi necessário compreender em profundidade a física e o comportamento do giroscópio e do acelerômetro.

7. FOTOS

